Assignment 3

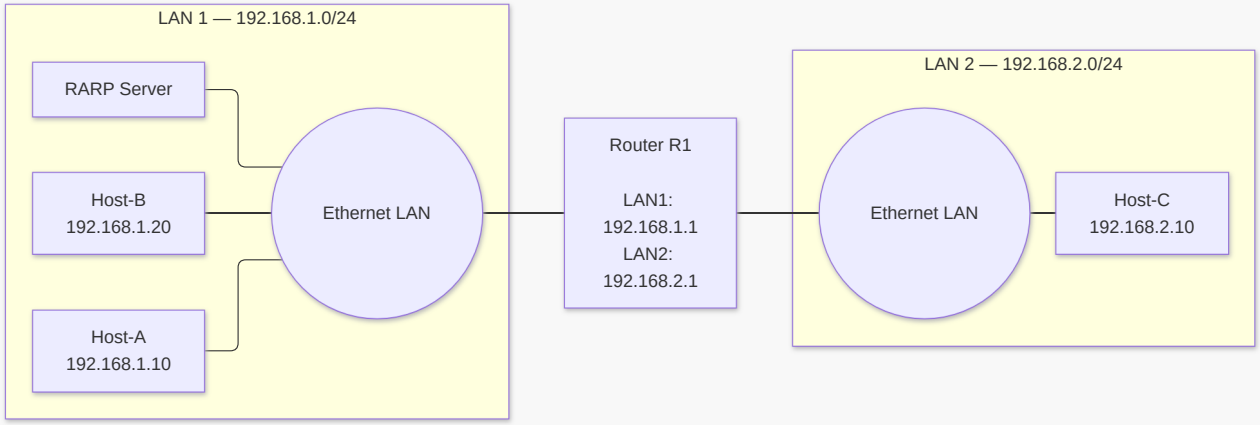
```
Name : Ravish Ranjan
Roll no. : 45
Course : MCA
Subject : Data Communications and Computer Networks
```

Design and implement a Python program to simulate the working of the Reverse Address Resolution Protocol (RARP) and the Address Resolution Protocol (ARP) over a simulated network consisting of two Ethernet LAN connected by a router, and demonstrate how these protocols enable actual IP packet delivery — across subnets via a default gateway.

Objectives:

1. Model a shared broadcast network (Network) to which multiple hosts can be attached, each
2. Simulate the RARP resolution process first, since a host that knows only its own MAC address cannot participate in ARP at all until it has an IP:
    - The host broadcasts a RARP Request containing its own MAC address.
    - A dedicated RARP server, maintaining a static MAC-to-IP mapping table, replies with the corresponding IP if a match exists.
    - Demonstrate both a successful RARP resolution (registered MAC) and a failure case (unregistered MAC, no reply). Display the message appropriately.
3. Simulate the ARP resolution process for hosts that already have an IP address:
    - Check the local ARP cache first; use the cached mapping on a hit.
    - On a cache miss, broadcast an ARP Request to the local segment; only the host owning the target IP responds with a unicast ARP Reply.
    - Cache the resolved mapping with a TTL, and demonstrate cache expiry forcing re-resolution.
    - Demonstrate an ARP resolution failure for an IP address with no corresponding host.
4. Implement actual IP packet sending (send_packet) on each host, which:
    - Determines whether the destination IP lies on the sender's own subnet or a remote one.
    - For a same-subnet destination, resolves the destination's MAC directly via ARP and delivers the packet with no router involvement.
    - For a different-subnet destination, resolves the default gateway's MAC via ARP (a purely local lookup) and hands the packet to the gateway instead.
5. Implement a Router connecting two LAN segments, where each router interface:
    - Behaves like a host for ARP purposes (correctly answers "who-has" request).
    - On receiving a forwarded IP packet, decrements its TTL, determines the correct outbound interface for the destination subnet, performs a fresh, independent ARP resolution on that outbound packet, and re-frames the packet with its own MAC as the new source before transmitting it onward and finally to the host.

Log each step of every transaction — broadcast vs. unicast delivery, packet fields, cache hits/misses/expiry, routing decisions, and final delivery — so the full protocol behavior is traceable end to end.




```
import ipaddress
import time
from dataclasses import dataclass

def divider(title):
    print("\n" + "=" * 75)
    print(title)
    print("=" * 75)

@dataclass
class ARPCacheEntry:
    ip: str
    mac: str
    expires_at: float


class ARPCache:
    def __init__(self, owner_name):
        self.owner_name = owner_name
        self.entries = {}

    def lookup(self, ip):
        entry = self.entries.get(ip)

        if entry is None:
            print(f"[ARP CACHE] {self.owner_name}: MISS for {ip}")
            return None

        if time.monotonic() >= entry.expires_at:
            print(f"[ARP CACHE] {self.owner_name}: EXPIRED entry {ip} ->
{entry.mac}")
            del self.entries[ip]
            return None
```

```
        remaining = entry.expires_at - time.monotonic()

        print(f"[ARP CACHE] {self.owner_name}: HIT {ip} -> {entry.mac} (TTL
remaining: {remaining:.1f}s)")
        return entry.mac

    def add(self, ip, mac, ttl):
        self.entries[ip] = ARPCacheEntry(
            ip=ip,
            mac=mac,
            expires_at=time.monotonic() + ttl
        )
        print(f"[ARP CACHE] {self.owner_name}: Cached {ip} -> {mac} for
{ttl}s")

    def clear(self):
        self.entries.clear()

@dataclass
class EthernetFrame:
    src_mac: str
    dst_mac: str
    payload: object

@dataclass
class IPPacket:
    src_ip: str
    dst_ip: str
    ttl: int
    data: str

@dataclass
class ARPRequest:
    sender_ip: str
    sender_mac: str
    target_ip: str

@dataclass
class ARPReply:
    sender_ip: str
    sender_mac: str
    target_ip: str
    target_mac: str

@dataclass
class RARPRequest:
    sender_mac: str

@dataclass
class RARPReply:
    server_mac: str
    target_mac: str
    assigned_ip: str
```

```
class Network:
    """
    Simulates one Ethernet LAN / broadcast domain.

    Every device attached to this network can receive broadcasts.
    Unicast frames are delivered only to the device owning the
    destination MAC.
    """

    def __init__(self, name, network_address):
        self.name = name
        self.network = ipaddress.ip_network(network_address)
        self.devices = []

    def connect(self, device):
        self.devices.append(device)
        device.network = self

        print(f"[NETWORK] {device.name} attached to {self.name}
({self.network})")

    def is_local(self, ip):
        return ipaddress.ip_address(ip) in self.network

    def transmit(self, frame):
        print(f"[ETHERNET] {self.name}: {frame.src_mac} ->
{frame.dst_mac}")

        # Broadcast
        if frame.dst_mac == "FF:FF:FF:FF:FF:FF":
            print("[ETHERNET] Broadcast frame delivered to all devices")

            for device in self.devices:
                device.receive_frame(frame)

            return

        # Unicast
        for device in self.devices:
            if device.mac == frame.dst_mac:
                print(f"[ETHERNET] Unicast frame delivered to
{device.name}")
                device.receive_frame(frame)
                return

        print(f"[ETHERNET] No device owns MAC {frame.dst_mac}. Frame
dropped.")

class RARPServer:
    def __init__(self, name, mac):
        self.name = name
        self.mac = mac
        self.network = None
```

```
        # Static MAC -> IP mapping
        self.mapping = {}

    def add_mapping(self, mac, ip):
        self.mapping[mac] = ip

    def receive_frame(self, frame):
        if not isinstance(frame.payload, RARPRequest):
            return

        request = frame.payload

        print(f"[RARP SERVER] {self.name} received RARP Request")
        print(f"    Sender MAC : {request.sender_mac}")

        assigned_ip = self.mapping.get(request.sender_mac)

        if assigned_ip is None:
            print(f"[RARP SERVER] No mapping found for
{request.sender_mac}")
            print("[RARP SERVER] No RARP Reply sent")
            return

        print(f"[RARP SERVER] Mapping found: {request.sender_mac} ->
{assigned_ip}")

        reply = RARPReply(
            server_mac=self.mac,
            target_mac=request.sender_mac,
            assigned_ip=assigned_ip
        )

        frame = EthernetFrame(
            src_mac=self.mac,
            dst_mac=request.sender_mac,
            payload=reply
        )

        print("[RARP SERVER] Sending RARP Reply (UNICAST)")

        self.network.transmit(frame)

class Host:
    def __init__(
        self,
        name,
        mac,
        ip=None,
        subnet=None,
        gateway_ip=None,
        arp_ttl=10
    ):
        self.name = name
        self.mac = mac
```

```
        self.ip = ip
        self.subnet = ipaddress.ip_network(subnet) if subnet else None
        self.gateway_ip = gateway_ip
        self.arp_ttl = arp_ttl

        self.network = None
        self.arp_cache = ARPCache(name)

    def send_rarp_request(self):
        divider(f"RARP REQUEST: {self.name}")

        print(f"[RARP] {self.name} knows only its MAC address:")
        print(f"       MAC = {self.mac}")

        request = RARPRequest(sender_mac=self.mac)

        frame = EthernetFrame(
            src_mac=self.mac,
            dst_mac="FF:FF:FF:FF:FF:FF",
            payload=request
        )

        print("[RARP] Broadcasting RARP Request")
        print(f"       Sender MAC = {self.mac}")

        self.network.transmit(frame)

    def receive_rarp_reply(self, reply):
        print(f"[RARP] {self.name} received RARP Reply")

        if reply.target_mac != self.mac:
            return

        self.ip = reply.assigned_ip

        print(f"[RARP] SUCCESS: MAC {self.mac} assigned IP {self.ip}")

    def resolve_arp(self, target_ip):
        """
        Resolve an IP -> MAC mapping using the local LAN.
        """

        divider(f"ARP RESOLUTION: {self.name} -> {target_ip}")

        # Check cache first
        cached_mac = self.arp_cache.lookup(target_ip)

        if cached_mac:
            print(f"[ARP] Using cached mapping: {target_ip} ->
{cached_mac}")
            return cached_mac

        print(f"[ARP] Cache MISS. Broadcasting ARP Request on
{self.network.name}")
```

```
        request = ARPRequest(
            sender_ip=self.ip,
            sender_mac=self.mac,
            target_ip=target_ip
        )

        frame = EthernetFrame(
            src_mac=self.mac,
            dst_mac="FF:FF:FF:FF:FF:FF",
            payload=request
        )

        self.network.transmit(frame)

        # After the broadcast, check whether a reply was cached.
        mac = self.arp_cache.lookup(target_ip)

        if mac:
            print(f"[ARP] SUCCESS: {target_ip} -> {mac}")
            return mac

        print(f"[ARP] FAILURE: No host responded for {target_ip}")

        return None

    def receive_frame(self, frame):
        payload = frame.payload

        if isinstance(payload, ARPRequest):
            print(f"[ARP] {self.name} received ARP Request")
            print(f"    Who has IP       : {payload.target_ip}")
            print(f"    Request sender   : {payload.sender_ip}
({payload.sender_mac})")

            if self.ip != payload.target_ip:
                print(f"[ARP] {self.name}: target IP does not match my IP
({self.ip}). Ignoring.")
                return

            print(f"[ARP] {self.name} owns {payload.target_ip}")

            reply = ARPReply(
                sender_ip=self.ip,
                sender_mac=self.mac,
                target_ip=payload.sender_ip,
                target_mac=payload.sender_mac
            )

            reply_frame = EthernetFrame(
                src_mac=self.mac,
                dst_mac=payload.sender_mac,
                payload=reply
            )
```

```
            print(f"[ARP] {self.name}: Sending ARP Reply (UNICAST)")
            print(f"      {self.ip} is at {self.mac}")

            self.network.transmit(reply_frame)
        elif isinstance(payload, ARPReply):
            if payload.target_mac != self.mac:
                return

            print(f"[ARP] {self.name} received ARP Reply")
            print(f"    {payload.sender_ip} is at {payload.sender_mac}")

            self.arp_cache.add(
                payload.sender_ip,
                payload.sender_mac,
                self.arp_ttl
            )
        elif isinstance(payload, RARPReply):
            self.receive_rarp_reply(payload)
        elif isinstance(payload, IPPacket):
            self.receive_ip_packet(frame)

    def send_packet(self, destination_ip, data, ttl=8):
        divider(f"IP PACKET: {self.name} -> {destination_ip}")

        if self.ip is None:
            print(f"[IP] {self.name} has no IP address.")
            print("[IP] Cannot send packet.")
            return

        destination = ipaddress.ip_address(destination_ip)

        # Determine local vs remote
        if destination in self.subnet:
            print(f"[IP] Destination {destination_ip} is on the local
subnet {self.subnet}")
            print("[IP] Router is NOT required.")
            next_hop_ip = destination_ip
        else:
            print(f"[IP] Destination {destination_ip} is NOT on local
subnet {self.subnet}")
            print("[IP] Destination is remote.")
            print(f"[IP] Using default gateway {self.gateway_ip}")
            next_hop_ip = self.gateway_ip

        # Resolve next-hop MAC
        next_hop_mac = self.resolve_arp(next_hop_ip)

        if next_hop_mac is None:
            print("[IP] Cannot send packet: next-hop MAC could not be
resolved.")
            return

        packet = IPPacket(
```

```
            src_ip=self.ip,
            dst_ip=destination_ip,
            ttl=ttl,
            data=data
        )

        print("")
        print("[IP] Creating IP packet:")
        print(f"     Source IP      : {packet.src_ip}")
        print(f"     Destination IP : {packet.dst_ip}")
        print(f"     TTL            : {packet.ttl}")
        print(f"     Data           : {packet.data}")

        print("")
        print("[ETHERNET] Encapsulating IP packet:")
        print(f"     Source MAC     : {self.mac}")
        print(f"     Destination MAC: {next_hop_mac}")

        frame = EthernetFrame(
            src_mac=self.mac,
            dst_mac=next_hop_mac,
            payload=packet
        )

        print(f"[IP] Sending frame onto {self.network.name}")
        self.network.transmit(frame)

    def receive_ip_packet(self, frame):
        packet = frame.payload

        print(f"[IP] {self.name} received IP packet")
        print(f"     Ethernet src MAC : {frame.src_mac}")
        print(f"     Ethernet dst MAC : {frame.dst_mac}")
        print(f"     IP source        : {packet.src_ip}")
        print(f"     IP destination   : {packet.dst_ip}")
        print(f"     TTL              : {packet.ttl}")
        print(f"     Data              : {packet.data}")

        if packet.dst_ip == self.ip:
            print(f"[IP] FINAL DELIVERY: packet reached {self.name}")
            print(f"[IP] Payload = \"{packet.data}\"")
        else:
            print(f"[IP] Packet is not addressed to {self.name}.
Ignoring.")

class RouterInterface:
    """
    A router interface behaves like a host for ARP.
    """

    def __init__(self, router, name, mac, ip, subnet):
        self.router = router
        self.name = name
        self.mac = mac
```

```
        self.ip = ip
        self.subnet = ipaddress.ip_network(subnet)
        self.network = None

    def receive_frame(self, frame):
        payload = frame.payload

        if isinstance(payload, ARPRequest):
            print(f"[ROUTER ARP] Interface {self.name} received ARP
Request")
            print(f"    Who has: {payload.target_ip}")

            if payload.target_ip != self.ip:
                print(f"[ROUTER ARP] Target does not match interface IP
{self.ip}")
                return

            reply = ARPReply(
                sender_ip=self.ip,
                sender_mac=self.mac,
                target_ip=payload.sender_ip,
                target_mac=payload.sender_mac
            )

            frame = EthernetFrame(
                src_mac=self.mac,
                dst_mac=payload.sender_mac,
                payload=reply
            )

            print(f"[ROUTER ARP] {self.name} replying:")
            print(f"    {self.ip} is at {self.mac}")

            self.network.transmit(frame)
        elif isinstance(payload, IPPacket):
            self.router.receive_packet(self, frame)

class Router:
    def __init__(self, name="R1"):
        self.name = name
        self.interfaces = []

    def add_interface(self, name, mac, ip, subnet, network):
        interface = RouterInterface(
            router=self,
            name=name,
            mac=mac,
            ip=ip,
            subnet=subnet
        )

        network.connect(interface)
        self.interfaces.append(interface)
```

```
        return interface

    def get_interface_for_network(self, network):
        for interface in self.interfaces:
            if network == interface.subnet:
                return interface
        return None

    def find_outbound_interface(self, destination_ip):
        destination = ipaddress.ip_address(destination_ip)
        for interface in self.interfaces:
            if destination in interface.subnet:
                return interface
        return None

    def receive_packet(self, inbound_interface, frame):
        packet = frame.payload

        divider(f"ROUTER {self.name}: FORWARDING PACKET")
        print(f"[ROUTER] Packet arrived on interface
{inbound_interface.name}")
        print(f"    Incoming Ethernet source MAC : {frame.src_mac}")
        print(f"    Incoming Ethernet dest MAC   : {frame.dst_mac}")
        print(f"    IP source      : {packet.src_ip}")
        print(f"    IP destination : {packet.dst_ip}")
        print(f"    Incoming TTL   : {packet.ttl}")

        packet.ttl -= 1

        print(f"[ROUTER] Decremented TTL -> {packet.ttl}")

        if packet.ttl <= 0:
            print("[ROUTER] TTL expired. Packet dropped.")
            return

        outbound_interface = self.find_outbound_interface(packet.dst_ip)

        if outbound_interface is None:
            print(f"[ROUTER] No route for {packet.dst_ip}. Packet
dropped.")
            return

        print(f"[ROUTER] Routing decision:")
        print(f"    Destination {packet.dst_ip} belongs to
{outbound_interface.subnet}")
        print(f"    Outbound interface = {outbound_interface.name}")

        if outbound_interface is inbound_interface:
            print("[ROUTER] Destination is on the incoming LAN.")
        else:
            print("[ROUTER] Packet must cross to another LAN.")

        target_mac =
self.router_arp_resolve(outbound_interface,packet.dst_ip)
```

```
        if target_mac is None:
            print("[ROUTER] ARP failed on outbound LAN.")
            print("[ROUTER] Packet dropped.")
            return

        print("")
        print("[ROUTER] Re-framing IP packet for outbound LAN:")
        print(f"    New Ethernet source MAC = {outbound_interface.mac}")
        print(f"    New Ethernet destination MAC = {target_mac}")
        print(f"    IP source remains = {packet.src_ip}")
        print(f"    IP destination remains = {packet.dst_ip}")
        print(f"    TTL after routing = {packet.ttl}")

        new_frame = EthernetFrame(
            src_mac=outbound_interface.mac,
            dst_mac=target_mac,
            payload=packet
        )

        print(f"[ROUTER] Transmitting packet on
{outbound_interface.network.name}")
        outbound_interface.network.transmit(new_frame)

    def router_arp_resolve(self, interface, target_ip):
        divider(f"ROUTER ARP: {interface.name} -> {target_ip}")
        print(f"[ROUTER ARP] No cached entry.")
        print(f"[ROUTER ARP] Broadcasting ARP Request on
{interface.network.name}")

        request = ARPRequest(
            sender_ip=interface.ip,
            sender_mac=interface.mac,
            target_ip=target_ip
        )

        frame = EthernetFrame(
            src_mac=interface.mac,
            dst_mac="FF:FF:FF:FF:FF:FF",
            payload=request
        )

        # Temporary mechanism for this simulation:
        # the router remembers ARP replies generated during
        # this resolution.
        interface._pending_arp = None
        original_receive = interface.router.receive_router_arp_reply

        # Broadcast
        interface.network.transmit(frame)

        # Check result
        if interface._pending_arp is not None:
            mac = interface._pending_arp
```

```
                print(f"[ROUTER ARP] SUCCESS: {target_ip} -> {mac}")
                return mac

            print(f"[ROUTER ARP] FAILURE: No response for {target_ip}")
            return None

        def receive_router_arp_reply(self, interface, reply):
            interface._pending_arp = reply.sender_mac

    original_interface_receive = RouterInterface.receive_frame

    def router_interface_receive_frame(self, frame):
        payload = frame.payload

        if isinstance(payload, ARPReply):
            print(f"[ROUTER ARP] Interface {self.name} received ARP Reply")

            if payload.target_mac != self.mac:
                return

            print(f"[ROUTER ARP] {payload.sender_ip} is at
    {payload.sender_mac}")

            self.router.receive_router_arp_reply(self,payload)
            return

        original_interface_receive(self, frame)

    RouterInterface.receive_frame = router_interface_receive_frame

    def main():
        divider("CREATING SIMULATED NETWORK")

        lan1 = Network(
            "LAN-1",
            "192.168.1.0/24"
        )

        lan2 = Network(
            "LAN-2",
            "192.168.2.0/24"
        )

        rarp_server = RARPServer(
            "RARP-Server",
            "AA:AA:AA:AA:AA:AA"
        )

        lan1.connect(rarp_server)

        host_a = Host(
            name="Host-A",
            mac="00:00:00:00:00:01",
            ip="192.168.1.10",
```

```
        subnet="192.168.1.0/24",
        gateway_ip="192.168.1.1",
        arp_ttl=5
    )

    host_b = Host(
        name="Host-B",
        mac="00:00:00:00:00:02",
        ip="192.168.1.20",
        subnet="192.168.1.0/24",
        gateway_ip="192.168.1.1",
        arp_ttl=5
    )

    host_c = Host(
        name="Host-C",
        mac="00:00:00:00:00:03",
        ip="192.168.2.10",
        subnet="192.168.2.0/24",
        gateway_ip="192.168.2.1",
        arp_ttl=5
    )

    # Attach hosts
    lan1.connect(host_a)
    lan1.connect(host_b)
    lan2.connect(host_c)

    rarp_server.add_mapping(
        "00:00:00:00:00:01",
        "192.168.1.10"
    )

    router = Router("R1")

    router_if1 = router.add_interface(
        name="R1-LAN1",
        mac="00:AA:00:00:00:01",
        ip="192.168.1.1",
        subnet="192.168.1.0/24",
        network=lan1
    )

    router_if2 = router.add_interface(
        name="R1-LAN2",
        mac="00:AA:00:00:00:02",
        ip="192.168.2.1",
        subnet="192.168.2.0/24",
        network=lan2
    )

    divider("1. SUCCESSFUL RARP RESOLUTION")

    host_a.ip = None
```

```
    host_a.send_rarp_request()

    # Restore subnet/gateway after RARP
    host_a.subnet = ipaddress.ip_network("192.168.1.0/24")
    host_a.gateway_ip = "192.168.1.1"

    divider("2. FAILED RARP RESOLUTION")

    unknown_host = Host(
        name="Unknown-Host",
        mac="00:00:00:00:99:99"
    )

    lan1.connect(unknown_host)

    unknown_host.send_rarp_request()

    divider("3. SAME-SUBNET COMMUNICATION")

    host_a.send_packet(
        destination_ip="192.168.1.20",
        data="Hello Host-B",
        ttl=8
    )

    divider("4. ARP CACHE HIT")

    host_a.send_packet(
        destination_ip="192.168.1.20",
        data="Second message using cached ARP",
        ttl=8
    )

    divider("5. ARP CACHE EXPIRY")

    print("[DEMO] Waiting for ARP cache TTL to expire...")

    time.sleep(6)

    host_a.send_packet(
        destination_ip="192.168.1.20",
        data="ARP must be resolved again",
        ttl=8
    )

    divider("6. ARP FAILURE")

    host_a.send_packet(
        destination_ip="192.168.1.99",
        data="This packet cannot be delivered",
        ttl=8
    )
```

```
    divider("7. CROSS-SUBNET COMMUNICATION")

    host_a.send_packet(
        destination_ip="192.168.2.10",
        data="Hello Host-C across the router",
        ttl=8
    )

    divider("8. CROSS-SUBNET SECOND PACKET")

    host_a.send_packet(
        destination_ip="192.168.2.10",
        data="Second packet to Host-C",
        ttl=8
    )

    divider("9. TTL DEMONSTRATION")

    host_a.send_packet(
        destination_ip="192.168.2.10",
        data="Packet with TTL=1",
        ttl=1
    )

    divider("SIMULATION COMPLETE")

if __name__ == "__main__":
    main()
```

Output

```
ravish@ravishPC:../cn/prog ⎇ (main)> py ./a3.py

================================================================================
CREATING SIMULATED NETWORK
================================================================================
[NETWORK] RARP-Server attached to LAN-1 (192.168.1.0/24)
[NETWORK] Host-A attached to LAN-1 (192.168.1.0/24)
[NETWORK] Host-B attached to LAN-1 (192.168.1.0/24)
[NETWORK] Host-C attached to LAN-2 (192.168.2.0/24)
[NETWORK] R1-LAN1 attached to LAN-1 (192.168.1.0/24)
[NETWORK] R1-LAN2 attached to LAN-2 (192.168.2.0/24)

================================================================================
1. SUCCESSFUL RARP RESOLUTION
================================================================================

================================================================================
RARP REQUEST: Host-A
================================================================================
[RARP] Host-A knows only its MAC address:
```

```
       MAC = 00:00:00:00:00:01
[RARP] Broadcasting RARP Request
       Sender MAC = 00:00:00:00:00:01
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> FF:FF:FF:FF:FF:FF
[ETHERNET] Broadcast frame delivered to all devices
[RARP SERVER] RARP-Server received RARP Request
    Sender MAC : 00:00:00:00:00:01
[RARP SERVER] Mapping found: 00:00:00:00:00:01 -> 192.168.1.10
[RARP SERVER] Sending RARP Reply (UNICAST)
[ETHERNET] LAN-1: AA:AA:AA:AA:AA:AA -> 00:00:00:00:00:01
[ETHERNET] Unicast frame delivered to Host-A
[RARP] Host-A received RARP Reply
[RARP] SUCCESS: MAC 00:00:00:00:00:01 assigned IP 192.168.1.10

================================================================================
2. FAILED RARP RESOLUTION
================================================================================
[NETWORK] Unknown-Host attached to LAN-1 (192.168.1.0/24)

================================================================================
RARP REQUEST: Unknown-Host
================================================================================
[RARP] Unknown-Host knows only its MAC address:
       MAC = 00:00:00:00:99:99
[RARP] Broadcasting RARP Request
       Sender MAC = 00:00:00:00:99:99
[ETHERNET] LAN-1: 00:00:00:00:99:99 -> FF:FF:FF:FF:FF:FF
[ETHERNET] Broadcast frame delivered to all devices
[RARP SERVER] RARP-Server received RARP Request
    Sender MAC : 00:00:00:00:99:99
[RARP SERVER] No mapping found for 00:00:00:00:99:99
[RARP SERVER] No RARP Reply sent

================================================================================
3. SAME-SUBNET COMMUNICATION
================================================================================

================================================================================
IP PACKET: Host-A -> 192.168.1.20
================================================================================
[IP] Destination 192.168.1.20 is on the local subnet 192.168.1.0/24
[IP] Router is NOT required.

================================================================================
ARP RESOLUTION: Host-A -> 192.168.1.20
================================================================================
[ARP CACHE] Host-A: MISS for 192.168.1.20
[ARP] Cache MISS. Broadcasting ARP Request on LAN-1
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> FF:FF:FF:FF:FF:FF
[ETHERNET] Broadcast frame delivered to all devices
[ARP] Host-A received ARP Request
    Who has IP       : 192.168.1.20
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Host-A: target IP does not match my IP (192.168.1.10). Ignoring.
```

```
[ARP] Host-B received ARP Request
    Who has IP       : 192.168.1.20
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Host-B owns 192.168.1.20
[ARP] Host-B: Sending ARP Reply (UNICAST)
      192.168.1.20 is at 00:00:00:00:00:02
[ETHERNET] LAN-1: 00:00:00:00:00:02 -> 00:00:00:00:00:01
[ETHERNET] Unicast frame delivered to Host-A
[ARP] Host-A received ARP Reply
    192.168.1.20 is at 00:00:00:00:00:02
[ARP CACHE] Host-A: Cached 192.168.1.20 -> 00:00:00:00:00:02 for 5s
[ROUTER ARP] Interface R1-LAN1 received ARP Request
    Who has: 192.168.1.20
[ROUTER ARP] Target does not match interface IP 192.168.1.1
[ARP] Unknown-Host received ARP Request
    Who has IP       : 192.168.1.20
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Unknown-Host: target IP does not match my IP (None). Ignoring.
[ARP CACHE] Host-A: HIT 192.168.1.20 -> 00:00:00:00:00:02 (TTL remaining:
5.0s)
[ARP] SUCCESS: 192.168.1.20 -> 00:00:00:00:00:02

[IP] Creating IP packet:
     Source IP      : 192.168.1.10
     Destination IP : 192.168.1.20
     TTL            : 8
     Data           : Hello Host-B

[ETHERNET] Encapsulating IP packet:
     Source MAC     : 00:00:00:00:00:01
     Destination MAC: 00:00:00:00:00:02
[IP] Sending frame onto LAN-1
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> 00:00:00:00:00:02
[ETHERNET] Unicast frame delivered to Host-B
[IP] Host-B received IP packet
     Ethernet src MAC : 00:00:00:00:00:01
     Ethernet dst MAC : 00:00:00:00:00:02
     IP source        : 192.168.1.10
     IP destination   : 192.168.1.20
     TTL              : 8
     Data              : Hello Host-B
[IP] FINAL DELIVERY: packet reached Host-B
[IP] Payload = "Hello Host-B"

================================================================================
4. ARP CACHE HIT
================================================================================

================================================================================
IP PACKET: Host-A -> 192.168.1.20
================================================================================
[IP] Destination 192.168.1.20 is on the local subnet 192.168.1.0/24
[IP] Router is NOT required.
```

```
================================================================================
ARP RESOLUTION: Host-A -> 192.168.1.20
================================================================================
[ARP CACHE] Host-A: HIT 192.168.1.20 -> 00:00:00:00:00:02 (TTL remaining:
5.0s)
[ARP] Using cached mapping: 192.168.1.20 -> 00:00:00:00:00:02

[IP] Creating IP packet:
     Source IP      : 192.168.1.10
     Destination IP : 192.168.1.20
     TTL            : 8
     Data           : Second message using cached ARP

[ETHERNET] Encapsulating IP packet:
     Source MAC     : 00:00:00:00:00:01
     Destination MAC: 00:00:00:00:00:02
[IP] Sending frame onto LAN-1
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> 00:00:00:00:00:02
[ETHERNET] Unicast frame delivered to Host-B
[IP] Host-B received IP packet
     Ethernet src MAC : 00:00:00:00:00:01
     Ethernet dst MAC : 00:00:00:00:00:02
     IP source        : 192.168.1.10
     IP destination   : 192.168.1.20
     TTL              : 8
     Data              : Second message using cached ARP
[IP] FINAL DELIVERY: packet reached Host-B
[IP] Payload = "Second message using cached ARP"

================================================================================
5. ARP CACHE EXPIRY
================================================================================
[DEMO] Waiting for ARP cache TTL to expire...

================================================================================
IP PACKET: Host-A -> 192.168.1.20
================================================================================
[IP] Destination 192.168.1.20 is on the local subnet 192.168.1.0/24
[IP] Router is NOT required.

================================================================================
ARP RESOLUTION: Host-A -> 192.168.1.20
================================================================================
[ARP CACHE] Host-A: EXPIRED entry 192.168.1.20 -> 00:00:00:00:00:02
[ARP] Cache MISS. Broadcasting ARP Request on LAN-1
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> FF:FF:FF:FF:FF:FF
[ETHERNET] Broadcast frame delivered to all devices
[ARP] Host-A received ARP Request
    Who has IP       : 192.168.1.20
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Host-A: target IP does not match my IP (192.168.1.10). Ignoring.
[ARP] Host-B received ARP Request
    Who has IP       : 192.168.1.20
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
```

```
[ARP] Host-B owns 192.168.1.20
[ARP] Host-B: Sending ARP Reply (UNICAST)
      192.168.1.20 is at 00:00:00:00:00:02
[ETHERNET] LAN-1: 00:00:00:00:00:02 -> 00:00:00:00:00:01
[ETHERNET] Unicast frame delivered to Host-A
[ARP] Host-A received ARP Reply
    192.168.1.20 is at 00:00:00:00:00:02
[ARP CACHE] Host-A: Cached 192.168.1.20 -> 00:00:00:00:00:02 for 5s
[ROUTER ARP] Interface R1-LAN1 received ARP Request
    Who has: 192.168.1.20
[ROUTER ARP] Target does not match interface IP 192.168.1.1
[ARP] Unknown-Host received ARP Request
    Who has IP       : 192.168.1.20
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Unknown-Host: target IP does not match my IP (None). Ignoring.
[ARP CACHE] Host-A: HIT 192.168.1.20 -> 00:00:00:00:00:02 (TTL remaining:
5.0s)
[ARP] SUCCESS: 192.168.1.20 -> 00:00:00:00:00:02

[IP] Creating IP packet:
     Source IP      : 192.168.1.10
     Destination IP : 192.168.1.20
     TTL            : 8
     Data           : ARP must be resolved again

[ETHERNET] Encapsulating IP packet:
     Source MAC     : 00:00:00:00:00:01
     Destination MAC: 00:00:00:00:00:02
[IP] Sending frame onto LAN-1
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> 00:00:00:00:00:02
[ETHERNET] Unicast frame delivered to Host-B
[IP] Host-B received IP packet
     Ethernet src MAC : 00:00:00:00:00:01
     Ethernet dst MAC : 00:00:00:00:00:02
     IP source        : 192.168.1.10
     IP destination   : 192.168.1.20
     TTL              : 8
     Data              : ARP must be resolved again
[IP] FINAL DELIVERY: packet reached Host-B
[IP] Payload = "ARP must be resolved again"

================================================================================
6. ARP FAILURE
================================================================================

================================================================================
IP PACKET: Host-A -> 192.168.1.99
================================================================================
[IP] Destination 192.168.1.99 is on the local subnet 192.168.1.0/24
[IP] Router is NOT required.

================================================================================
ARP RESOLUTION: Host-A -> 192.168.1.99
================================================================================
```

```
[ARP CACHE] Host-A: MISS for 192.168.1.99
[ARP] Cache MISS. Broadcasting ARP Request on LAN-1
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> FF:FF:FF:FF:FF:FF
[ETHERNET] Broadcast frame delivered to all devices
[ARP] Host-A received ARP Request
    Who has IP       : 192.168.1.99
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Host-A: target IP does not match my IP (192.168.1.10). Ignoring.
[ARP] Host-B received ARP Request
    Who has IP       : 192.168.1.99
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Host-B: target IP does not match my IP (192.168.1.20). Ignoring.
[ROUTER ARP] Interface R1-LAN1 received ARP Request
    Who has: 192.168.1.99
[ROUTER ARP] Target does not match interface IP 192.168.1.1
[ARP] Unknown-Host received ARP Request
    Who has IP       : 192.168.1.99
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Unknown-Host: target IP does not match my IP (None). Ignoring.
[ARP CACHE] Host-A: MISS for 192.168.1.99
[ARP] FAILURE: No host responded for 192.168.1.99
[IP] Cannot send packet: next-hop MAC could not be resolved.

================================================================================
7. CROSS-SUBNET COMMUNICATION
================================================================================

================================================================================
IP PACKET: Host-A -> 192.168.2.10
================================================================================
[IP] Destination 192.168.2.10 is NOT on local subnet 192.168.1.0/24
[IP] Destination is remote.
[IP] Using default gateway 192.168.1.1

================================================================================
ARP RESOLUTION: Host-A -> 192.168.1.1
================================================================================
[ARP CACHE] Host-A: MISS for 192.168.1.1
[ARP] Cache MISS. Broadcasting ARP Request on LAN-1
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> FF:FF:FF:FF:FF:FF
[ETHERNET] Broadcast frame delivered to all devices
[ARP] Host-A received ARP Request
    Who has IP       : 192.168.1.1
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Host-A: target IP does not match my IP (192.168.1.10). Ignoring.
[ARP] Host-B received ARP Request
    Who has IP       : 192.168.1.1
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Host-B: target IP does not match my IP (192.168.1.20). Ignoring.
[ROUTER ARP] Interface R1-LAN1 received ARP Request
    Who has: 192.168.1.1
[ROUTER ARP] R1-LAN1 replying:
    192.168.1.1 is at 00:AA:00:00:00:01
[ETHERNET] LAN-1: 00:AA:00:00:00:01 -> 00:00:00:00:00:01
```

```
[ETHERNET] Unicast frame delivered to Host-A
[ARP] Host-A received ARP Reply
    192.168.1.1 is at 00:AA:00:00:00:01
[ARP CACHE] Host-A: Cached 192.168.1.1 -> 00:AA:00:00:00:01 for 5s
[ARP] Unknown-Host received ARP Request
    Who has IP       : 192.168.1.1
    Request sender   : 192.168.1.10 (00:00:00:00:00:01)
[ARP] Unknown-Host: target IP does not match my IP (None). Ignoring.
[ARP CACHE] Host-A: HIT 192.168.1.1 -> 00:AA:00:00:00:01 (TTL remaining:
5.0s)
[ARP] SUCCESS: 192.168.1.1 -> 00:AA:00:00:00:01

[IP] Creating IP packet:
     Source IP      : 192.168.1.10
     Destination IP : 192.168.2.10
     TTL            : 8
     Data           : Hello Host-C across the router

[ETHERNET] Encapsulating IP packet:
     Source MAC     : 00:00:00:00:00:01
     Destination MAC: 00:AA:00:00:00:01
[IP] Sending frame onto LAN-1
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> 00:AA:00:00:00:01
[ETHERNET] Unicast frame delivered to R1-LAN1

================================================================================
ROUTER R1: FORWARDING PACKET
================================================================================
[ROUTER] Packet arrived on interface R1-LAN1
    Incoming Ethernet source MAC : 00:00:00:00:00:01
    Incoming Ethernet dest MAC   : 00:AA:00:00:00:01
    IP source      : 192.168.1.10
    IP destination : 192.168.2.10
    Incoming TTL   : 8
[ROUTER] Decremented TTL -> 7
[ROUTER] Routing decision:
    Destination 192.168.2.10 belongs to 192.168.2.0/24
    Outbound interface = R1-LAN2
[ROUTER] Packet must cross to another LAN.

================================================================================
ROUTER ARP: R1-LAN2 -> 192.168.2.10
================================================================================
[ROUTER ARP] No cached entry.
[ROUTER ARP] Broadcasting ARP Request on LAN-2
[ETHERNET] LAN-2: 00:AA:00:00:00:02 -> FF:FF:FF:FF:FF:FF
[ETHERNET] Broadcast frame delivered to all devices
[ARP] Host-C received ARP Request
    Who has IP       : 192.168.2.10
    Request sender   : 192.168.2.1 (00:AA:00:00:00:02)
[ARP] Host-C owns 192.168.2.10
[ARP] Host-C: Sending ARP Reply (UNICAST)
      192.168.2.10 is at 00:00:00:00:00:03
[ETHERNET] LAN-2: 00:00:00:00:00:03 -> 00:AA:00:00:00:02
```

```
[ETHERNET] Unicast frame delivered to R1-LAN2
[ROUTER ARP] Interface R1-LAN2 received ARP Reply
[ROUTER ARP] 192.168.2.10 is at 00:00:00:00:00:03
[ROUTER ARP] Interface R1-LAN2 received ARP Request
    Who has: 192.168.2.10
[ROUTER ARP] Target does not match interface IP 192.168.2.1
[ROUTER ARP] SUCCESS: 192.168.2.10 -> 00:00:00:00:00:03

[ROUTER] Re-framing IP packet for outbound LAN:
    New Ethernet source MAC = 00:AA:00:00:00:02
    New Ethernet destination MAC = 00:00:00:00:00:03
    IP source remains = 192.168.1.10
    IP destination remains = 192.168.2.10
    TTL after routing = 7
[ROUTER] Transmitting packet on LAN-2
[ETHERNET] LAN-2: 00:AA:00:00:00:02 -> 00:00:00:00:00:03
[ETHERNET] Unicast frame delivered to Host-C
[IP] Host-C received IP packet
     Ethernet src MAC : 00:AA:00:00:00:02
     Ethernet dst MAC : 00:00:00:00:00:03
     IP source        : 192.168.1.10
     IP destination   : 192.168.2.10
     TTL              : 7
     Data              : Hello Host-C across the router
[IP] FINAL DELIVERY: packet reached Host-C
[IP] Payload = "Hello Host-C across the router"

================================================================================
8. CROSS-SUBNET SECOND PACKET
================================================================================

================================================================================
IP PACKET: Host-A -> 192.168.2.10
================================================================================
[IP] Destination 192.168.2.10 is NOT on local subnet 192.168.1.0/24
[IP] Destination is remote.
[IP] Using default gateway 192.168.1.1

================================================================================
ARP RESOLUTION: Host-A -> 192.168.1.1
================================================================================
[ARP CACHE] Host-A: HIT 192.168.1.1 -> 00:AA:00:00:00:01 (TTL remaining:
5.0s)
[ARP] Using cached mapping: 192.168.1.1 -> 00:AA:00:00:00:01

[IP] Creating IP packet:
     Source IP      : 192.168.1.10
     Destination IP : 192.168.2.10
     TTL            : 8
     Data           : Second packet to Host-C

[ETHERNET] Encapsulating IP packet:
     Source MAC     : 00:00:00:00:00:01
     Destination MAC: 00:AA:00:00:00:01
```

```
[IP] Sending frame onto LAN-1
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> 00:AA:00:00:00:01
[ETHERNET] Unicast frame delivered to R1-LAN1

================================================================================
ROUTER R1: FORWARDING PACKET
================================================================================
[ROUTER] Packet arrived on interface R1-LAN1
    Incoming Ethernet source MAC : 00:00:00:00:00:01
    Incoming Ethernet dest MAC   : 00:AA:00:00:00:01
    IP source      : 192.168.1.10
    IP destination : 192.168.2.10
    Incoming TTL   : 8
[ROUTER] Decremented TTL -> 7
[ROUTER] Routing decision:
    Destination 192.168.2.10 belongs to 192.168.2.0/24
    Outbound interface = R1-LAN2
[ROUTER] Packet must cross to another LAN.

================================================================================
ROUTER ARP: R1-LAN2 -> 192.168.2.10
================================================================================
[ROUTER ARP] No cached entry.
[ROUTER ARP] Broadcasting ARP Request on LAN-2
[ETHERNET] LAN-2: 00:AA:00:00:00:02 -> FF:FF:FF:FF:FF:FF
[ETHERNET] Broadcast frame delivered to all devices
[ARP] Host-C received ARP Request
    Who has IP       : 192.168.2.10
    Request sender   : 192.168.2.1 (00:AA:00:00:00:02)
[ARP] Host-C owns 192.168.2.10
[ARP] Host-C: Sending ARP Reply (UNICAST)
      192.168.2.10 is at 00:00:00:00:00:03
[ETHERNET] LAN-2: 00:00:00:00:00:03 -> 00:AA:00:00:00:02
[ETHERNET] Unicast frame delivered to R1-LAN2
[ROUTER ARP] Interface R1-LAN2 received ARP Reply
[ROUTER ARP] 192.168.2.10 is at 00:00:00:00:00:03
[ROUTER ARP] Interface R1-LAN2 received ARP Request
    Who has: 192.168.2.10
[ROUTER ARP] Target does not match interface IP 192.168.2.1
[ROUTER ARP] SUCCESS: 192.168.2.10 -> 00:00:00:00:00:03

[ROUTER] Re-framing IP packet for outbound LAN:
    New Ethernet source MAC = 00:AA:00:00:00:02
    New Ethernet destination MAC = 00:00:00:00:00:03
    IP source remains = 192.168.1.10
    IP destination remains = 192.168.2.10
    TTL after routing = 7
[ROUTER] Transmitting packet on LAN-2
[ETHERNET] LAN-2: 00:AA:00:00:00:02 -> 00:00:00:00:00:03
[ETHERNET] Unicast frame delivered to Host-C
[IP] Host-C received IP packet
     Ethernet src MAC : 00:AA:00:00:00:02
     Ethernet dst MAC : 00:00:00:00:00:03
     IP source        : 192.168.1.10
```

```
     IP destination   : 192.168.2.10
     TTL              : 7
     Data              : Second packet to Host-C
[IP] FINAL DELIVERY: packet reached Host-C
[IP] Payload = "Second packet to Host-C"

================================================================================
9. TTL DEMONSTRATION
================================================================================

================================================================================
IP PACKET: Host-A -> 192.168.2.10
================================================================================
[IP] Destination 192.168.2.10 is NOT on local subnet 192.168.1.0/24
[IP] Destination is remote.
[IP] Using default gateway 192.168.1.1

================================================================================
ARP RESOLUTION: Host-A -> 192.168.1.1
================================================================================
[ARP CACHE] Host-A: HIT 192.168.1.1 -> 00:AA:00:00:00:01 (TTL remaining:
5.0s)
[ARP] Using cached mapping: 192.168.1.1 -> 00:AA:00:00:00:01

[IP] Creating IP packet:
     Source IP      : 192.168.1.10
     Destination IP : 192.168.2.10
     TTL            : 1
     Data           : Packet with TTL=1

[ETHERNET] Encapsulating IP packet:
     Source MAC     : 00:00:00:00:00:01
     Destination MAC: 00:AA:00:00:00:01
[IP] Sending frame onto LAN-1
[ETHERNET] LAN-1: 00:00:00:00:00:01 -> 00:AA:00:00:00:01
[ETHERNET] Unicast frame delivered to R1-LAN1

================================================================================
ROUTER R1: FORWARDING PACKET
================================================================================
[ROUTER] Packet arrived on interface R1-LAN1
    Incoming Ethernet source MAC : 00:00:00:00:00:01
    Incoming Ethernet dest MAC   : 00:AA:00:00:00:01
    IP source      : 192.168.1.10
    IP destination : 192.168.2.10
    Incoming TTL   : 1
[ROUTER] Decremented TTL -> 0
[ROUTER] TTL expired. Packet dropped.

================================================================================
SIMULATION COMPLETE
================================================================================
ravish@ravishPC:../cn/prog ⎇ (main)>
```